

Raport Tehnic Final: Metodologii de Dezvoltare Software

Proiect: Mocking Forge — Un Simulator de Crafting Asistat de AI

Echipa de Dezvoltare MDS

15 iunie 2026

Rezumat

Acest document detaliază procesul de dezvoltare a proiectului **Mocking Forge**, realizat în cadrul disciplinei *Metode de Dezvoltare Software* (MDS). Proiectul explorează sinergia dintre arhitecturile software moderne (.NET 9, React, FastAPI) și tehnologiile de Inteligență Artificială (LLM locale, sinteză vocală neurală). Raportul analizează evoluția proiectului prin prisma istoricului de versiuni (Git), evidențiind modul în care instrumentele de asistență AI au accelerat ciclul de viață al dezvoltării.

1 Introducere

Proiectul **Mocking Forge** este o aplicație web interactivă de tip puzzle/alchemy în care jucătorii combină elemente de bază pentru a descoperi concepte complexe. Punctul de diferențiere major este utilizarea unui **Model de Limbaj Mare (LLM)** rulat local (prin Ollama) pentru a genera rezultate în timp real pentru combinații care nu există într-o bază de date predefinită.

2 Analiza Procesului de Dezvoltare (Git History)

Dezvoltarea a fost structurată în mai multe faze logice, documentate prin peste 100 de commit-uri. Am analizat istoricul complet pentru a identifica momentele cheie.

2.1 Cronologia Principalelor Milestone-uri

Tabela 1: Analiza selectivă a istoricului de commit-uri

Hash	Data	Descriere Tehnică
956887a	2026-05-07	Scaffolding inițial: Configurarea structurii monorepo pentru Frontend (Vite), Backend (.NET) și AI Service.
e821bf3	2026-05-07	Persistență: Implementarea entităților de bază EF Core și configurarea contextului bazei de date.
9148f34	2026-05-07	DevOps: Configurarea pipeline-ului CI/CD în GitHub Actions cu suport pentru Postgres în mediul de testare.
b226562	2026-05-07	UI/UX: Implementarea sistemului de Drag and Drop (DnD) folosind biblioteci specializate pentru React.
4c152a8	2026-05-07	AI Agent: Primul agent de companion integrat cu Ollama pentru generarea de replici contextuale.
0b65119	2026-05-09	Engine: Motorul de crafting semi-deterministic care balansează rețetele fixe cu cele generate de AI.

Tabela 1: Analiza selectivă a istoricului de commit-uri

Hash	Data	Descriere Tehnică
e530018	2026-06-14	Audio: Integrarea engine-ului de voce "Orc-boss" (Piper TTS + FFmpeg audio manipulation).
f6fb0d8	2026-06-15	Game Design: Implementarea a 10 misiuni jucabile și a logicii de progresie a nivelurilor.
e6c0c18	2026-06-15	Optimizare: Adăugarea unui strat de cache pentru rețete pentru a reduce latența LLM-ului.
4d6e394	2026-06-15	Auth/UI: Finalizarea sistemului de înregistrare și personalizarea interfeței utilizatorului.

3 Utilizarea Instrumentelor AI în Dezvoltare

O componentă critică a metodologiei MDS a fost utilizarea asistenților AI (identificați prin tag-ul `codex` sau prin colaborare directă). Impactul acestora a fost vizibil în patru arii majore:

3.1 Generarea de Cod și Arhitectură

Asistentul AI a fost utilizat pentru a genera boilerplate-ul necesar pentru:

- API-uri Minimaliste în .NET 9.
- Configurații Docker Compose complexe care includ servicii GPU pentru LLM-uri.
- Serviciul de AI bazat pe FastAPI, incluzând logica asincronă de comunicare cu Ollama.

3.2 Automatizarea Testării și Debugging

Analiza commit-urilor arată o utilizare intensivă a AI-ului pentru:

- **Unit Testing:** Generarea mock-urilor pentru `apiClient` în Vitest (Commit `fb248f`).
- **Integration Testing:** Configurarea test-containerelor pentru Postgres.
- **Fix-uri rapide:** Identificarea și rezolvarea problemelor de sincronizare a fișierelor de lock (`package-lock.json`).

4 Integrarea AI ca Nucleu de Runtime

Mocking Forge nu doar folosește AI pentru a scrie cod, ci îl integrează ca parte vitală a software-ului final.

4.1 Componenta de Companion (Mascota Interactivă)

Aplicația include un companion care reacționează la acțiunile jucătorului.

```

1 var instructions = new List<string> {
2     "You are a sarcastic orc boss in a crafting puzzle game.",
3     "Write one short mocking comment for this gameplay moment.",
4     "Rules: one sentence, under 120 characters."
5 };

```

Listing 1: Prompt Builder logic pentru Companion

4.2 Sistemul Text-to-Speech (TTS) Orc-Boss

Pentru a crește imersiunea, replicile generate de AI sunt transformate în audio printr-un pipeline neural:

1. **Sinteză:** Utilizarea modelului Piper (`en_GB-alan-medium`).
2. **Post-procesare:** Aplicarea unui filtru complex FFmpeg pentru a scădea pitch-ul și a adăuga un efect de "grit" și "reverb" specific unei peșteri.

5 Concluzii și Lecții Învățate

Proiectul **Mocking Forge** demonstrează că metodele moderne de dezvoltare software pot beneficia enorm de pe urma integrării AI. Lecțiile principale extrase din analiza istoricului proiectului sunt:

1. **Viteză de dezvoltare:** AI-ul permite trecerea rapidă de la specificații la implementare verificabilă.
2. **Calitatea Arhitecturii:** Utilizarea asistenților forțează adesea o separare mai clară a responsabilităților (ex: AI logic în Python, Game logic în C#).
3. **Mentenabilitate:** Deși AI-ul generează cod rapid, revizia umană rămâne critică pentru detalii precum validarea input-ului și securitatea sesiunilor.